

# INTERNSHIP TASK REPORT

## CLI Expense Tracker

*Week-02 | Task 1 | Khansa Malik*

### 1. Objective

The objective of this task was to develop a Python command-line expense tracker that supports adding, listing, searching, filtering, summarizing, deleting, and exporting expenses while maintaining persistent storage and handling invalid or corrupted data safely.

### 2. Design and Architecture

The application is implemented as a dependency-free command-line tool using Python standard-library modules. Expense records are stored persistently in a JSON file as a list of objects containing an ID, date, amount, category, and description. Command-line operations are organized around clear actions such as add, list, search, summary, delete, categories, and export, with an optional file path for alternate ledgers and testing.

### 3. Key Implementation Features

The tracker validates that amounts are positive numbers, dates follow YYYY-MM-DD format, and category and description fields satisfy length requirements. Users can filter expenses by category, month, or date range, search descriptions and categories by keyword, calculate monthly and per-category totals, delete records by ID, and export the complete ledger to CSV.

### 4. Data Persistence and Error Handling

Missing or empty data files are treated as an empty tracker and are created when data is first saved. If the JSON file is corrupt, unreadable, or has an invalid top-level structure, the original file is backed up with a timestamped .corrupt.\*.bak suffix and the program continues with a fresh store. Invalid individual records are skipped with a warning, while invalid command inputs are rejected without writing changes.

### 5. Testing

The script was validated manually by exercising all supported commands and both normal and failure cases. Testing covered valid and invalid additions, listing with category/month/date-range filters, matching and non-matching searches, monthly and category summaries, deletion of existing and non-existent IDs, missing data files, corrupt JSON files, and valid JSON containing an invalid top-level structure.

### 6. Outcome

The completed project is a functional command-line expense tracker with persistent JSON storage, robust input validation, recovery from corrupted files, and CSV export. The optional --file argument also makes the application easier to test and reuse with multiple data files without changing the core implementation.

### 7. Conclusion

This task provided practical experience in command-line interface design, JSON-based persistence, validation, exception handling, filtering, aggregation, and CSV generation in Python. The next iteration could introduce automated unit tests, structured logging, stronger schema validation, and separation into multiple modules as the application grows.

**Command Summary**

| Command          | Purpose                                            |
|------------------|----------------------------------------------------|
| add / list       | Create and view expenses with optional filters     |
| search / summary | Find records and calculate monthly/category totals |
| delete / export  | Remove an expense or generate a CSV file           |
| categories       | List categories currently in use                   |